Capstone Project 1: Loan Default Prediction using Machine Learning, FastAPI & Azure MLFlow

Objective

The objective of this capstone project is to build a complete, production-ready machine learning system that predicts loan default risk. The project will cover model development using Scikit-learn, deployment through a REST API using FastAPI, and cloud-based MLOps implementation using MLflow on Microsoft Azure.

Dataset

Loan Default Dataset containing borrower details such as income, credit history, loan amount, employment status, and repayment behavior.
Target Variable: Loan_Status (0 / 1)

Problem Statement

Loan default prediction is a critical task for financial institutions to minimize risk and optimize lending decisions. Incorrect predictions can lead to financial losses or missed opportunities.

You are provided with a dataset containing historical loan data. Your task is to analyze the data and build classification models to predict whether a borrower will default on a loan.

You will implement and compare the following algorithms:

- Logistic Regression
- Decision Tree
- Random Forest
- Naive Bayes
- K-Nearest Neighbors (KNN)

The final solution will include:

- A trained and serialized model (.pkl)
- A REST API for inference
- Cloud deployment and lifecycle management using MLflow on Azure

Target Variable: Loan_Status (Default / Non-Default)

Section 1: Machine Learning Pipeline (Scikit-learn)

Capstone Tasks

1. Load the dataset into a Pandas DataFrame, inspect its structure, and summarize key statistics including data types, shape, and missing values.
2. Perform data cleaning by handling missing values, removing duplicates, and treating outliers where necessary.
3. Conduct exploratory data analysis (EDA) by visualizing feature distributions, analyzing relationships with the target variable, and generating correlation insights.
4. Prepare the dataset by encoding categorical variables, scaling numerical features, and defining feature matrix (X) and target variable (y).
5. Analyze class imbalance and apply techniques such as SMOTE or upsampling to improve model performance.

6. Split the dataset into training and testing sets, ensuring proper validation strategy.
7. Train multiple classification models including Logistic Regression, Decision Tree, Random Forest, Naive Bayes, and KNN on the training data.
8. Improve model performance by applying hyperparameter tuning using GridSearchCV, where optimal parameters for each algorithm are identified through cross-validation.
9. Evaluate model performance using Accuracy, Precision, Recall, and F1-score, along with Confusion Matrix and ROC-AUC curve for comparison.
10. Compare all models, identify the best-performing one, and analyze feature importance to interpret key drivers of loan default.
11. Serialize the final model using pickle or joblib, generate a .pkl file, and validate it by reloading and performing predictions.

Section 2: MLOps using FastAPI (REST API Deployment)

Objective

To operationalize the trained machine learning model by building a scalable REST API using FastAPI, enabling real-time predictions.

Capstone Tasks

1. Set up a REST API using **FastAPI** and run it with **Uvicorn**, integrating the trained .pkl model for real-time inference.
2. Design input and output schemas using Pydantic, and create a /predict endpoint that accepts JSON data, validates inputs, and returns predictions.
3. Test the API ensuring correct responses and handling of edge cases.

Section 3: Azure MLOps using MLflow

Objective

To implement a complete MLOps lifecycle using **MLflow** on **Microsoft Azure** for experiment tracking, model management, and deployment.

1. Set up an environment on **Microsoft Azure** by creating a Machine Learning workspace and configuring compute resources.
2. Integrate **MLflow** to track experiments by logging parameters, metrics, and artifacts across multiple model runs.
3. Compare experiments within MLflow to identify the best-performing model and register it in the Model Registry with proper version control.
4. Deploy the registered model as a REST endpoint on Azure, enabling real-time inference with scalable infrastructure.
5. Test the deployed endpoint using API calls, monitor performance, and implement logging to track usage and accuracy.
6. Extend the workflow by enabling model monitoring, retraining strategies, and optional CI/CD integration for automated deployment.

Conclusion

This project provides hands-on experience in building, deploying, and managing machine learning systems. By combining Scikit-learn, FastAPI, and MLflow on Azure, you will develop a production-grade solution aligned with modern industry practices.

Project Deliverables

1. Project Documentation

A comprehensive project report must be submitted covering all three sections of the capstone. The documentation should clearly explain the complete workflow starting from data loading to final deployment.

The report should include a detailed description of data preprocessing steps, exploratory data analysis, feature engineering, model building, evaluation, and model selection. It should also document the process of saving the trained model as a .pkl file and validating it.

For the deployment sections, the documentation must explain how the REST API was built using **FastAPI**, including request/response design, testing process, and deployment steps. Additionally, it should cover the implementation of MLOps using **MLflow** on **Microsoft Azure**, including experiment tracking, model registration, and deployment.

The report must include relevant screenshots such as:

- Dataset preview and preprocessing steps
- EDA visualizations and insights
- Model training and evaluation results
- Confusion matrix and ROC curves
- Saved model output (.pkl)
- FastAPI endpoints (Swagger UI / Postman testing)
- MLflow experiment tracking dashboard
- Azure deployment and endpoint testing

The document should be well-structured, properly formatted, and written in a professional manner suitable for academic or industry review.

---

2. Coding Files

All source code used in the project must be submitted in an organized and modular format. The codebase should include scripts or notebooks for data preprocessing, feature engineering, model training, and evaluation using Scikit-learn.

Separate files should be provided for the REST API implementation using **FastAPI**, including model loading, input validation, and prediction endpoints. Code related to MLflow integration and Azure deployment should also be included, demonstrating experiment tracking and model management.

The project should ideally follow a clean folder structure (e.g., data/, notebooks/, src/, models/, api/, mlops/) and include a requirements.txt file for dependency management. All code must be executable, properly commented, and reproducible.